Depth-First Search

Prof. Timothy Sun

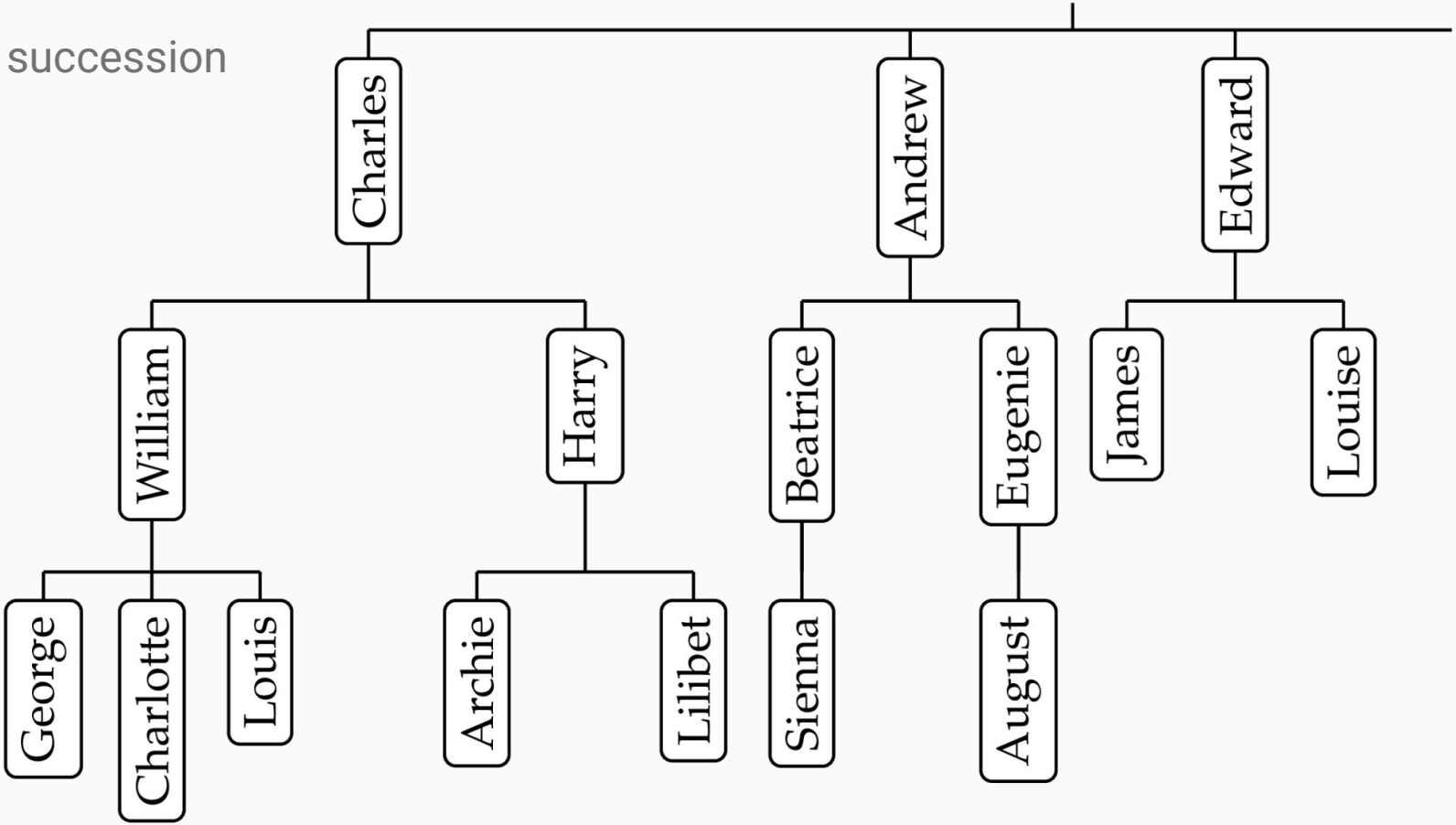
Depth-first search

DFS is especially useful because of the structure given by the **pre-order** and **post-order** traversals through the graph.

```
clock := 0
DFSAll(G):
    for each v ∈ V(G)
        if v is unmarked
            DFS(G, v)
DFS(G, v):
    mark v
    clock++, pre(v) := clock
    for each (v→w) ∈ E(G)
        if w is unmarked
            parent(w) := v
            DFS(G, w)
    clock++, post(v) := clock
```

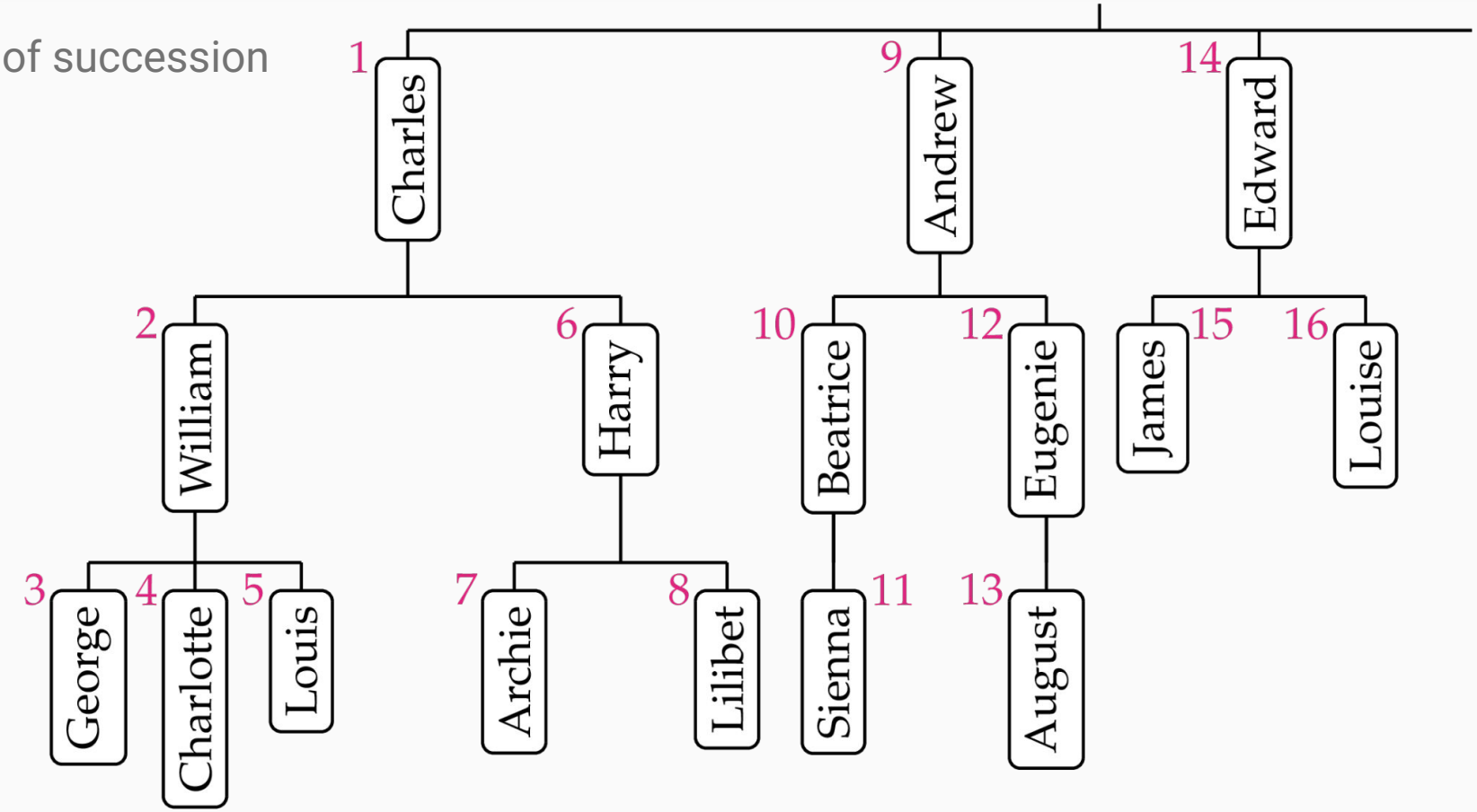
Pre-order traversal

UK line of succession



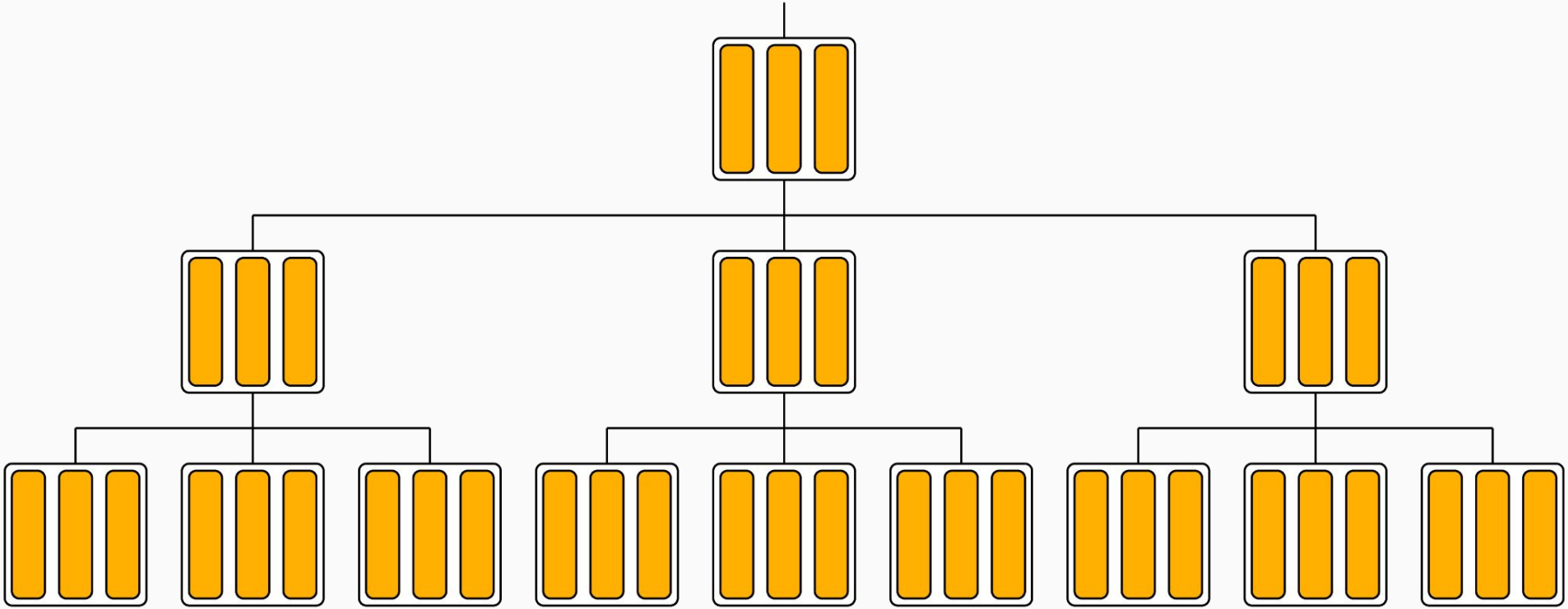
Pre-order traversal

UK line of succession



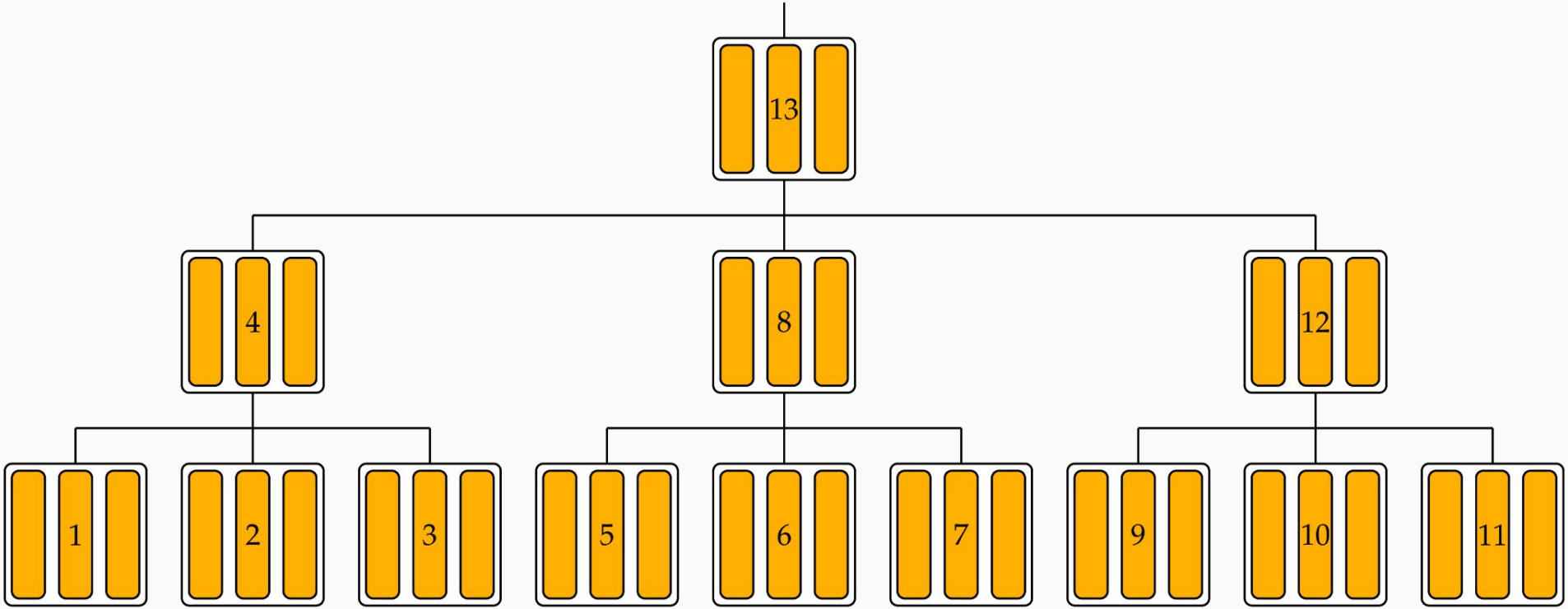
Post-order traversal

Celebrity Jeopardy! schedule



Post-order traversal

Celebrity Jeopardy! schedule



Depth-first search

DFS is especially useful because of the structure given by the **pre-order** and **post-order** traversals through the graph.

Just like connected components, we try doing DFS on every vertex, so every vertex is visited at some point.

$clock := 0$

$\boxed{\begin{array}{l} \text{DFSALL}(G): \\ \quad \textbf{for each } v \in V(G) \\ \quad\quad \textbf{if } v \text{ is unmarked} \\ \quad\quad\quad \text{DFS}(G, v) \end{array}}$

$\text{DFS}(G, v)$:

$\quad$ mark v

$\quad$ $clock{+}{+}, pre(v) := clock$

$\quad$ **for each** $(v \to w) \in E(G)$

$\quad\quad$ **if** w is unmarked

$\quad\quad\quad$ $parent(w) := v$

$\quad\quad\quad$ $\text{DFS}(G, w)$

$\quad$ $clock{+}{+}, post(v) := clock$

Depth-first search

A “clock” keeps track of when a vertex is on the recursion stack:

- **New:** clock < pre(***v***)
- **Active**: pre(***v***) < clock < post(***v***)
- **Finished**: post(***v***) < clock

$clock := 0$

$\text{DFSALL}(G)$:

  for each $v \in V(G)$

    if v is unmarked

      $\text{DFS}(G, v)$

$\text{DFS}(G, v)$:

  mark v

  $\boxed{clock{+}{+},\ pre(v) := clock}$

  for each $(v \rightarrow w) \in E(G)$

    if w is unmarked

      $parent(w) := v$

      $\text{DFS}(G, w)$

  $\boxed{clock{+}{+},\ post(v) := clock}$

Depth-first search

A “clock” keeps track of when a vertex is on the recursion stack:

- **New:** clock < pre(***v***)
- **Active**: pre(***v***) < clock < post(***v***)
- **Finished**: post(***v***) < clock

All DFSes in this class will be done in alphabetical order.

$clock := 0$

$\text{DFSALL}(G)$:

  $\boxed{\textbf{for each } v \in V(G)}$

    if v is unmarked

      $\text{DFS}(G, v)$

$\text{DFS}(G, v)$:

  mark v

  $clock{+}{+}, pre(v) := clock$

  $\boxed{\textbf{for each } (v \rightarrow w) \in E(G)}$

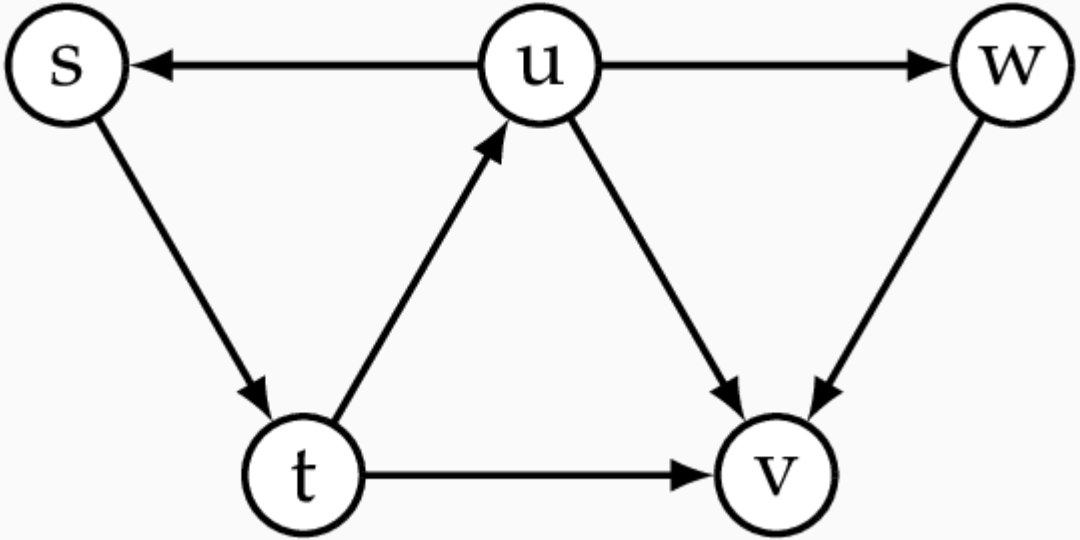
    if w is unmarked

      $parent(w) := v$

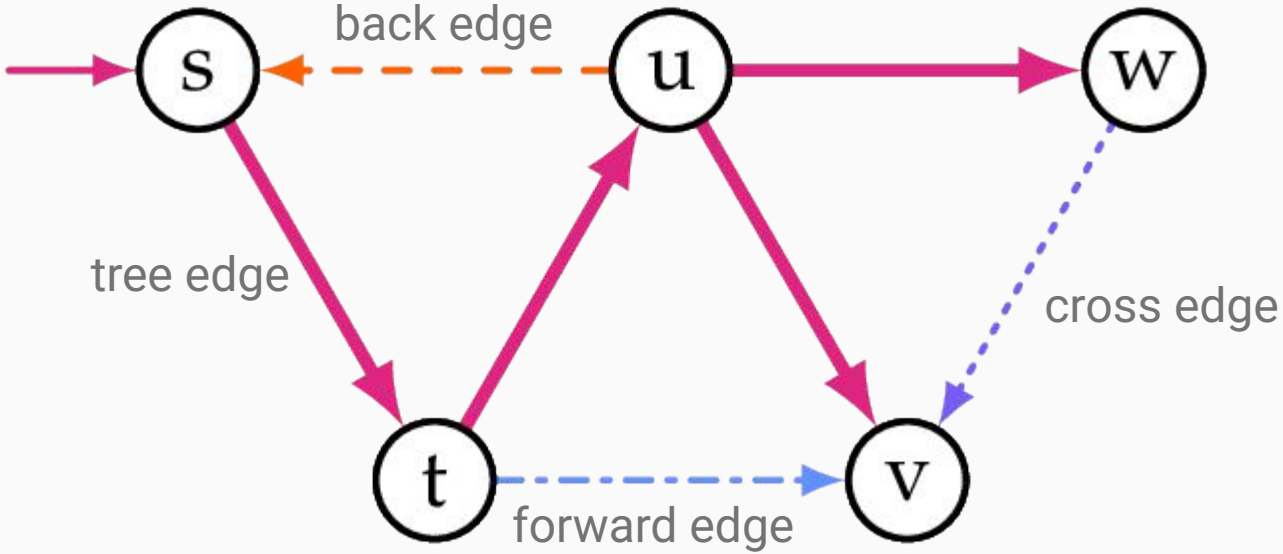
      $\text{DFS}(G, w)$

  $clock{+}{+}, post(v) := clock$

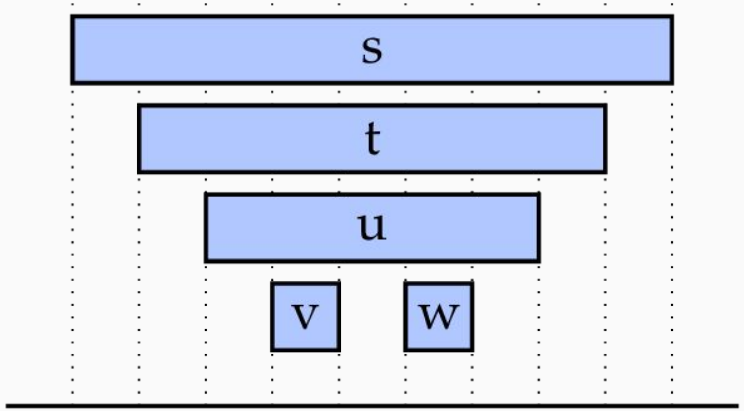
Depth-first search

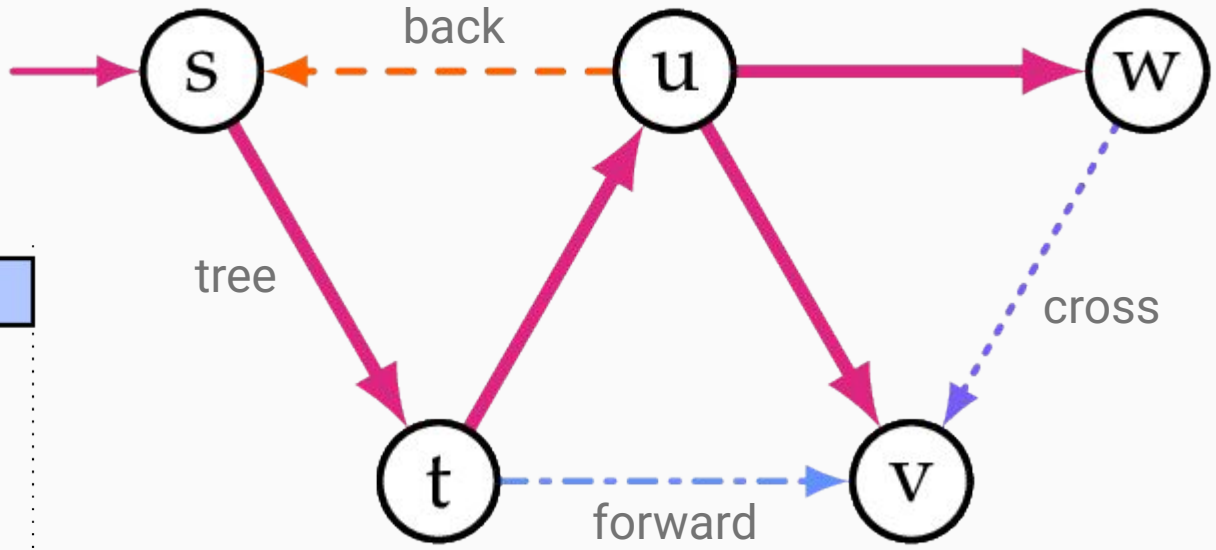


Depth-first search

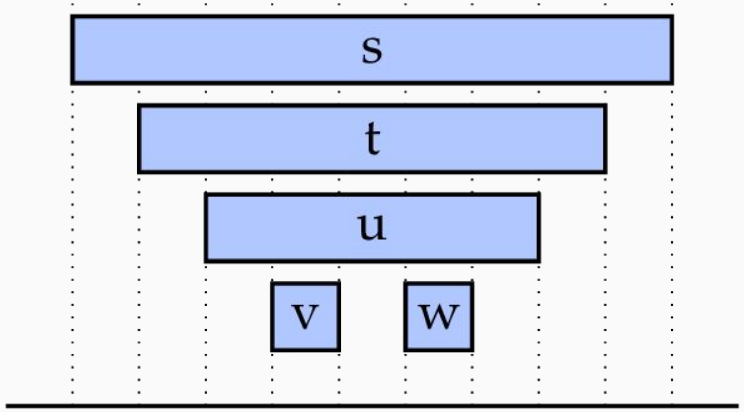


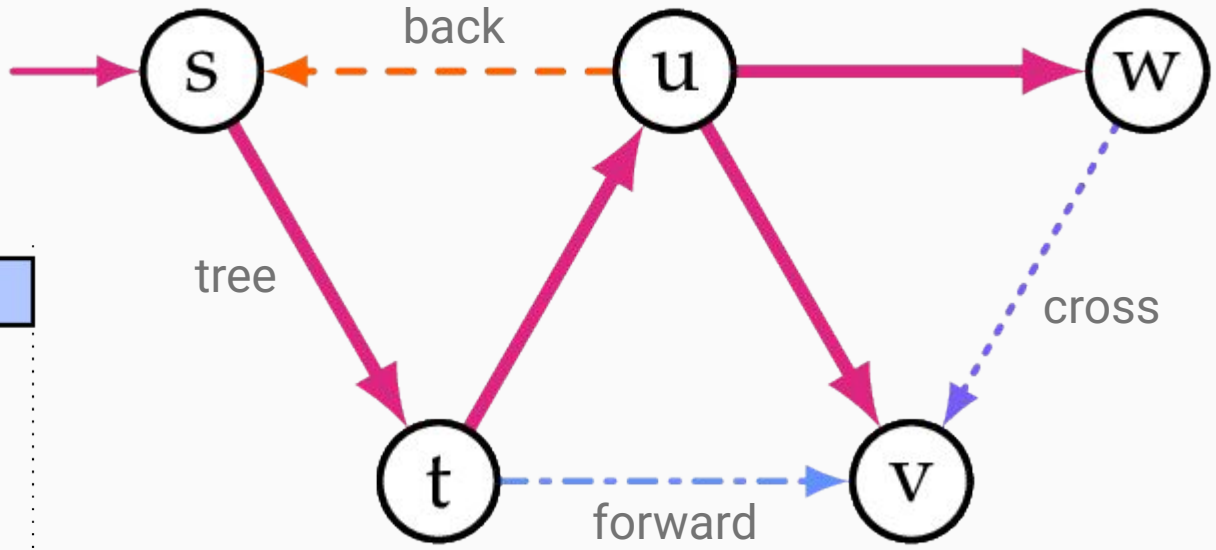
Depth-first search





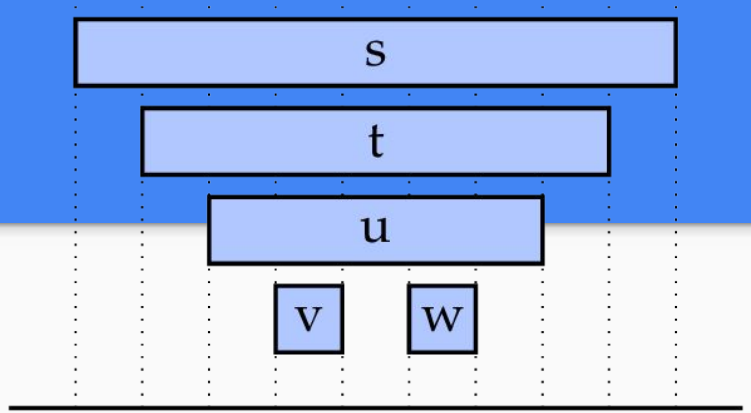
Depth-first search





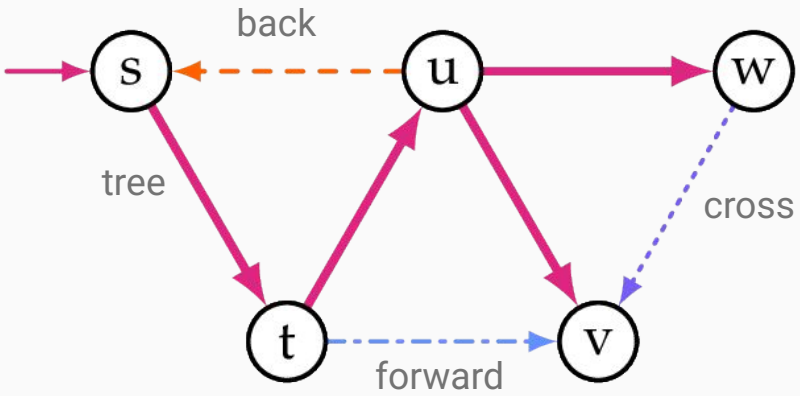
intervals are ***nesting***, or ***disjoint***

Parentheses Theorem

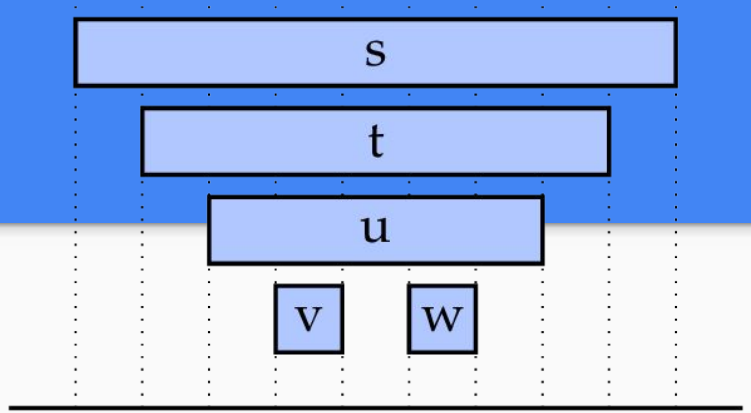


Back edge (e.g., $u \rightarrow s$)

- $[u] \subseteq [s]$
  - pre(s) < pre(u)
  - post(s) > post(u)

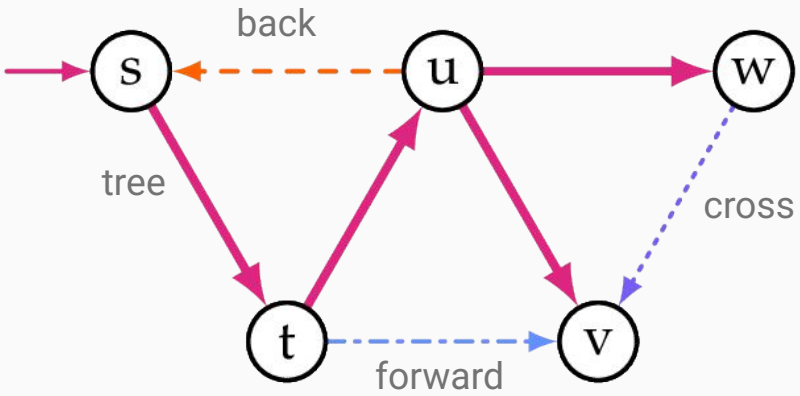


Parentheses Theorem

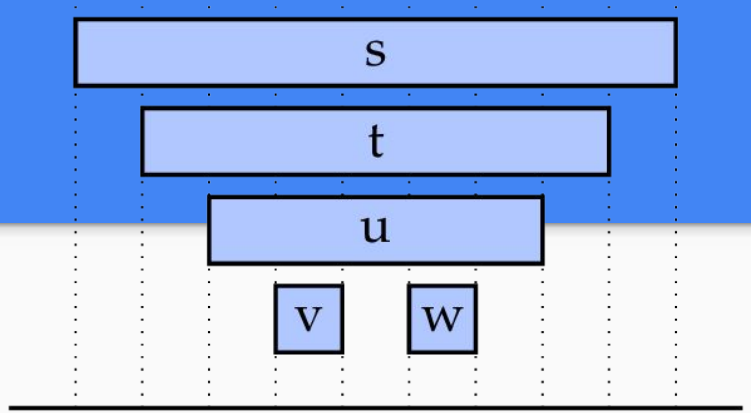


Tree edge (e.g., $s \rightarrow t$)

- $[t] \subseteq [s]$
  - pre(s) < pre(t)
  - post(s) > post(t)
- parent(t) = s

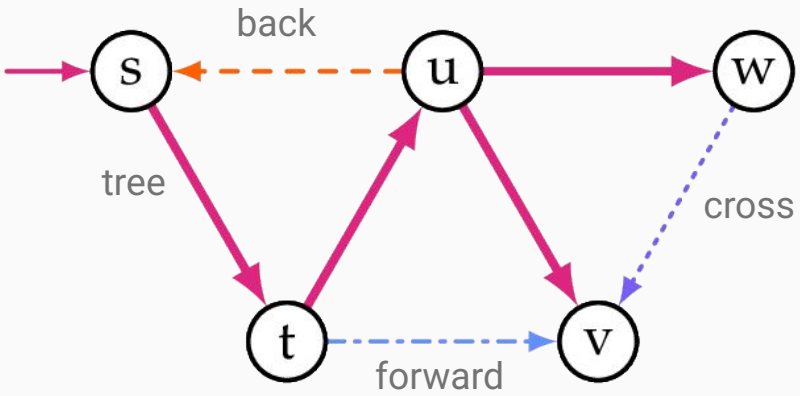


Parentheses Theorem

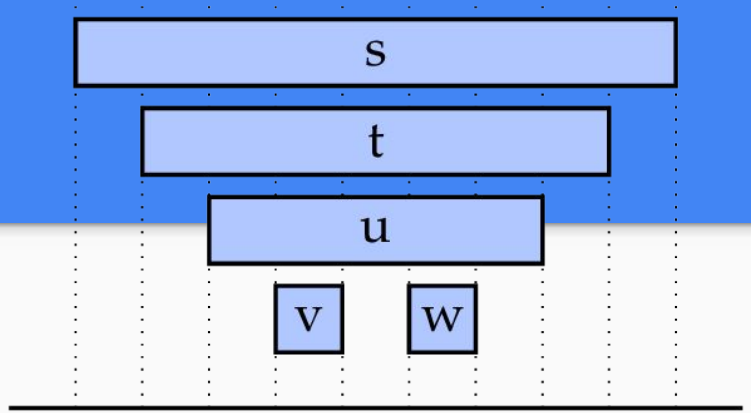


Forward edge (e.g., $t \rightarrow v$)

- $[v] \subseteq [t]$
  - pre(t) < pre(v)
  - post(t) > post(v)
- parent(v) ≠ t

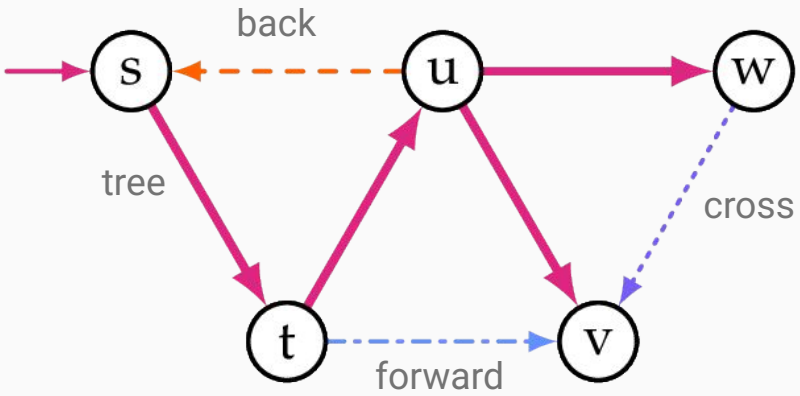


Parentheses Theorem

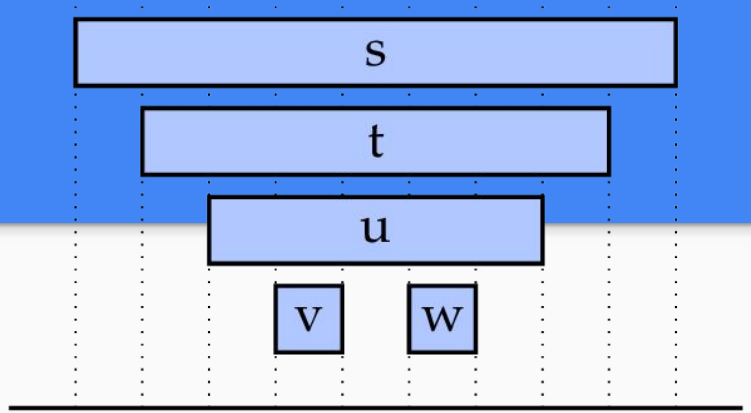


Cross edge (e.g., $w \rightarrow v$)

- [v] before [w]
  - pre(v) < post(v) < pre(w) < post(w)

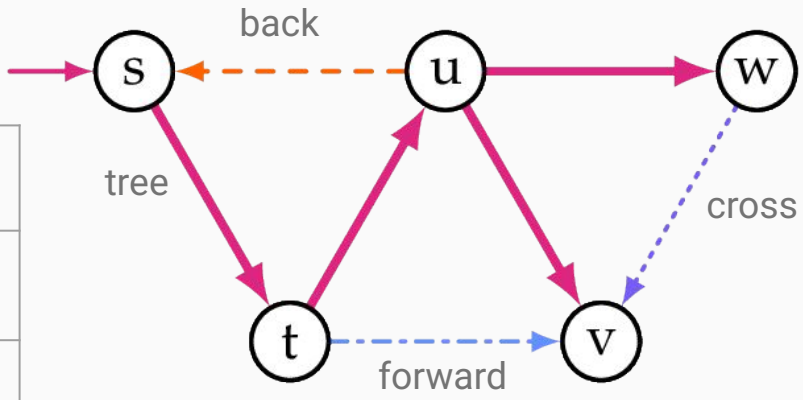


Parentheses Theorem

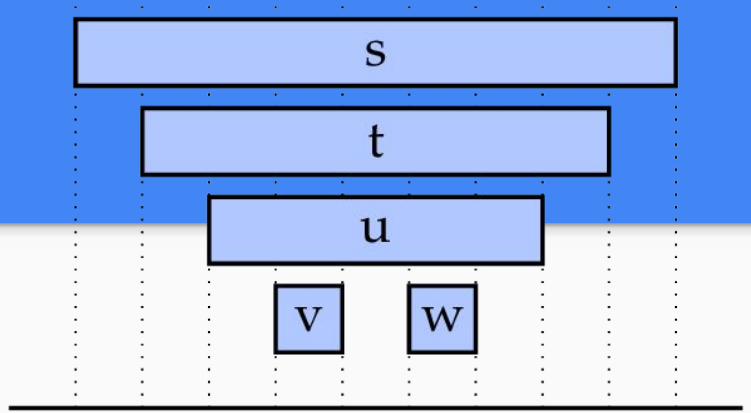


For a directed edge (***a***→***b***),

back	[***a***] ⊆ [***b***]	post(***a***) < post(***b***)
tree/forward	[***b***] ⊆ [***a***]	pre(***a***) < pre(***b***)
cross	[***b***] before [***a***]	otherwise

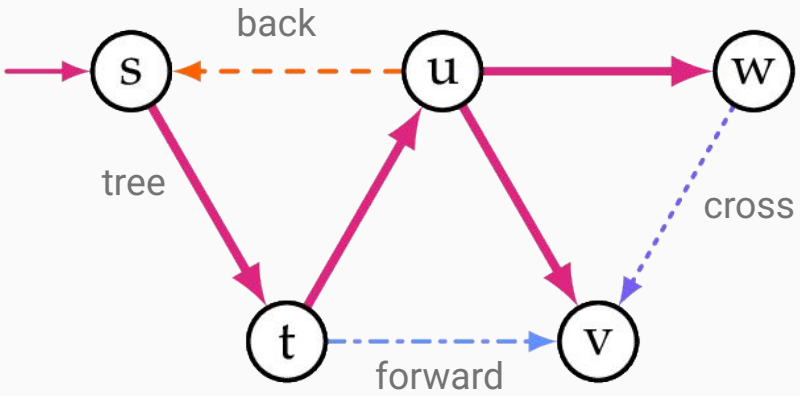


Parentheses Theorem

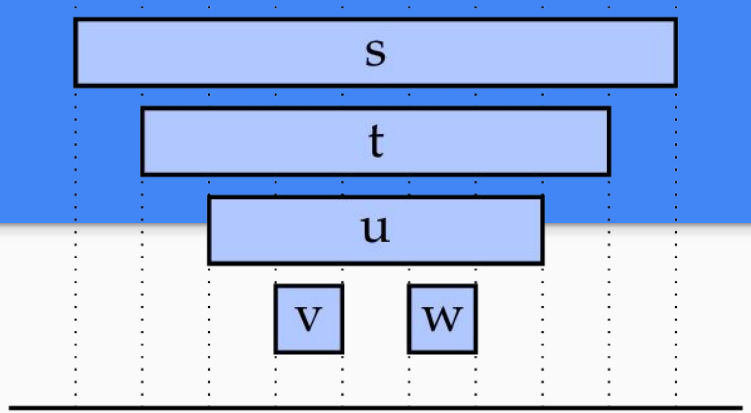


What about the fourth possibility?

- ***a***→***b***: [***a***] before [***b***]



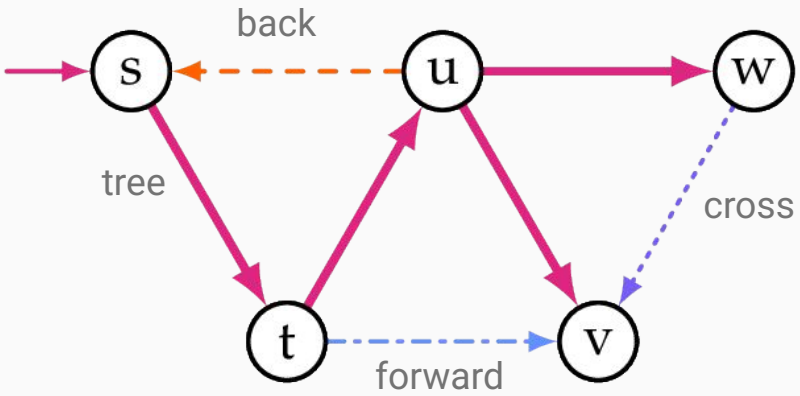
Parentheses Theorem



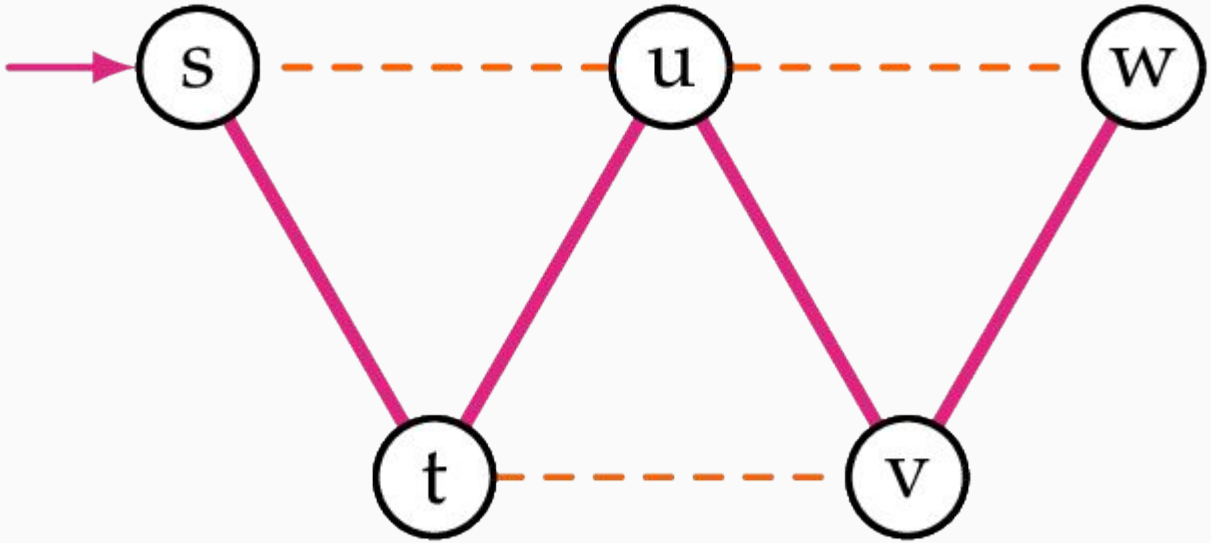
What about the fourth possibility?

- ***a***→***b***: [***a***] before [***b***]

Not possible!

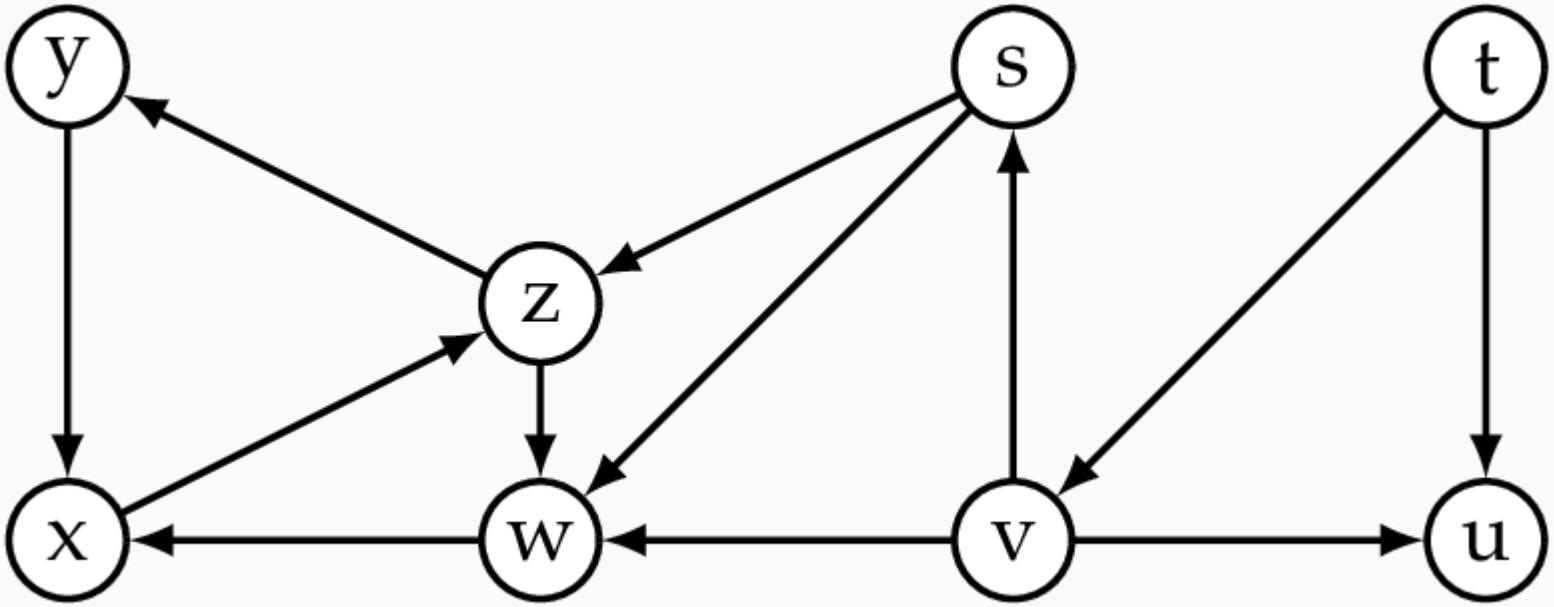


Undirected DFS

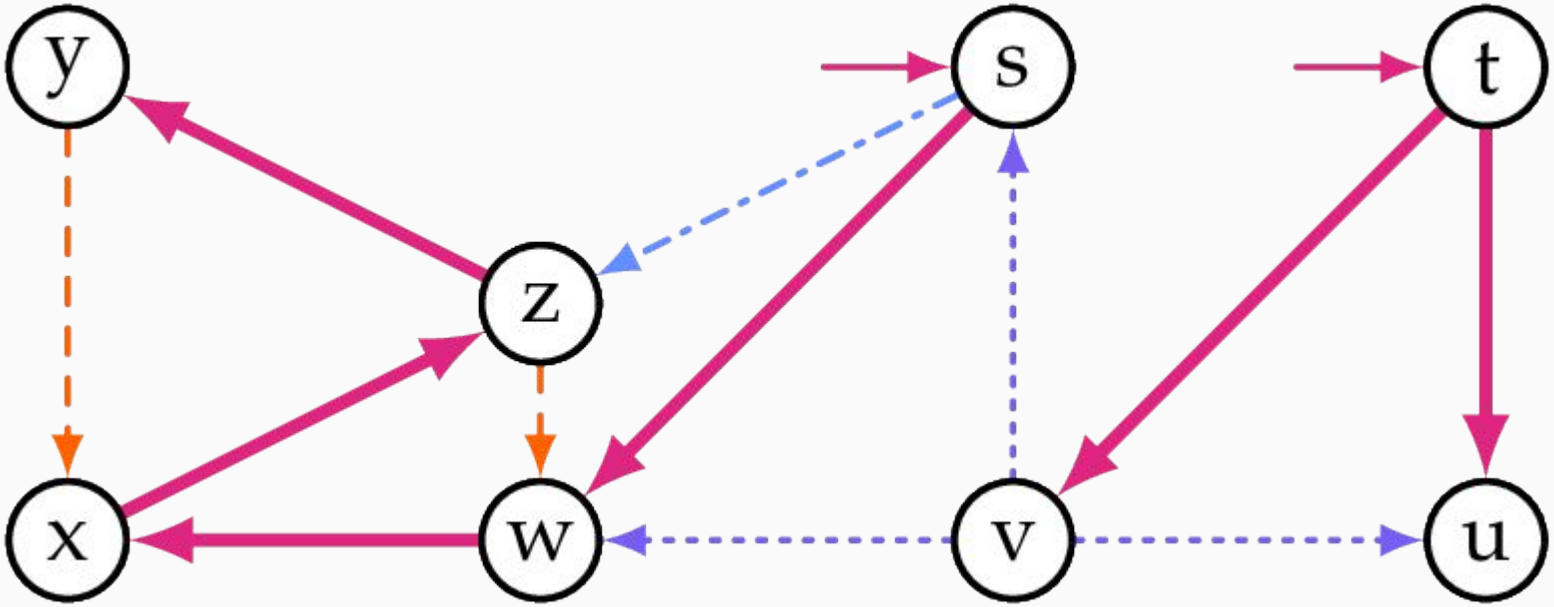


only tree and back edges

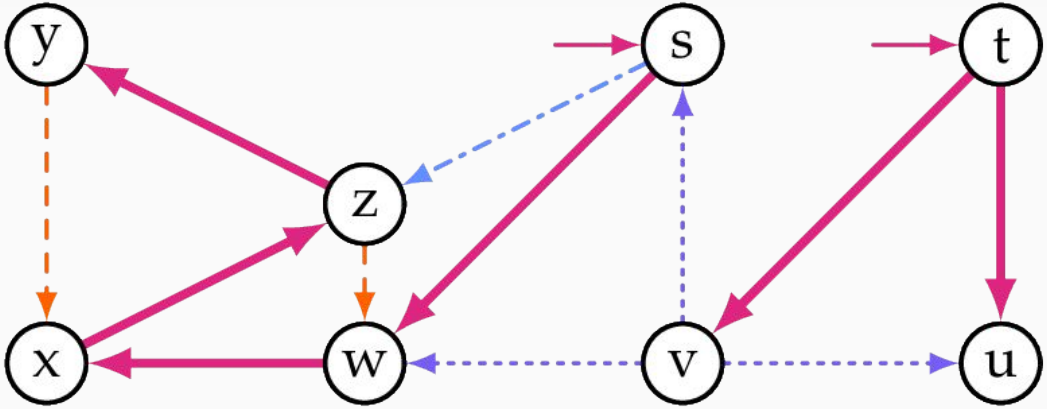
Some more examples

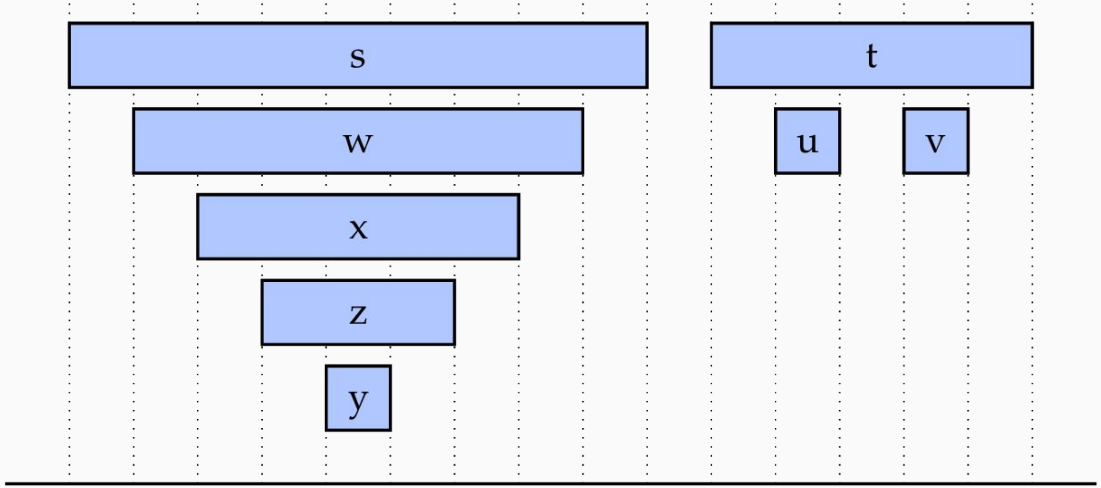


Some more examples

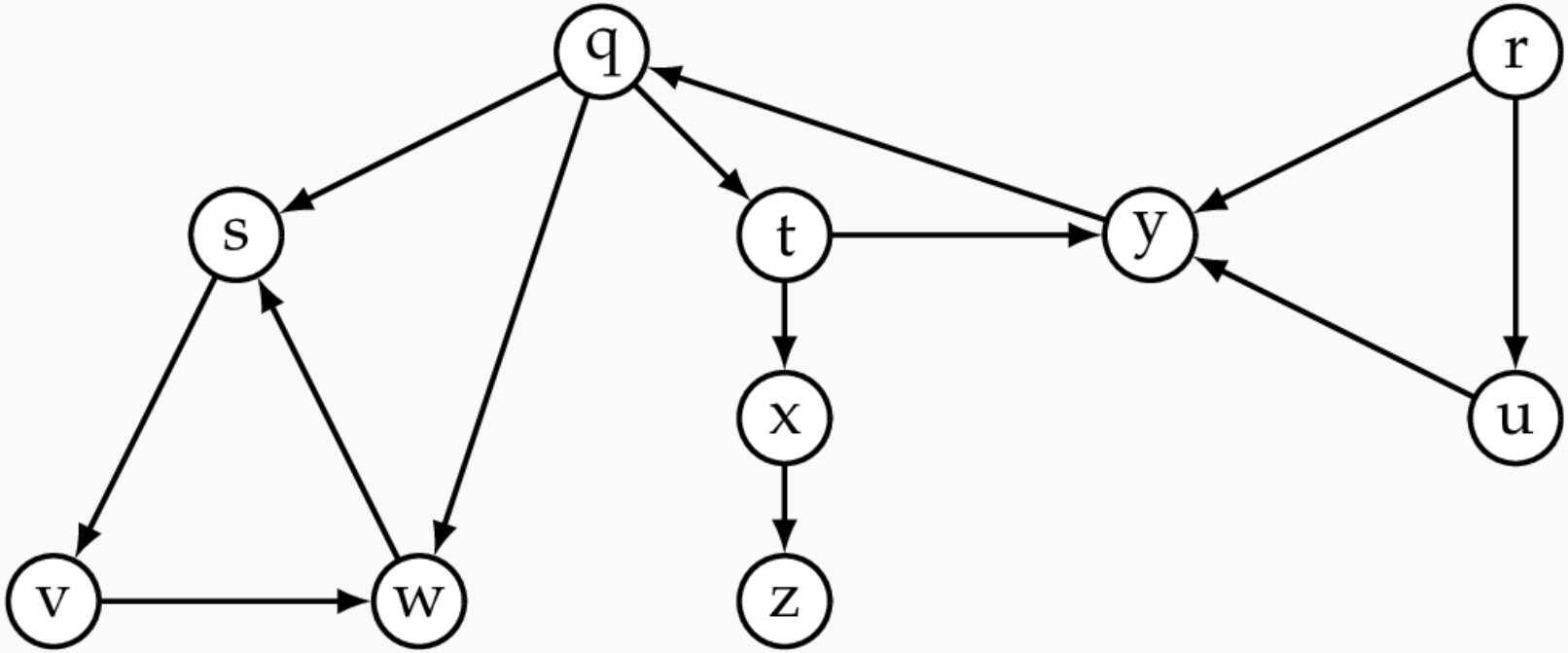


Some more examples

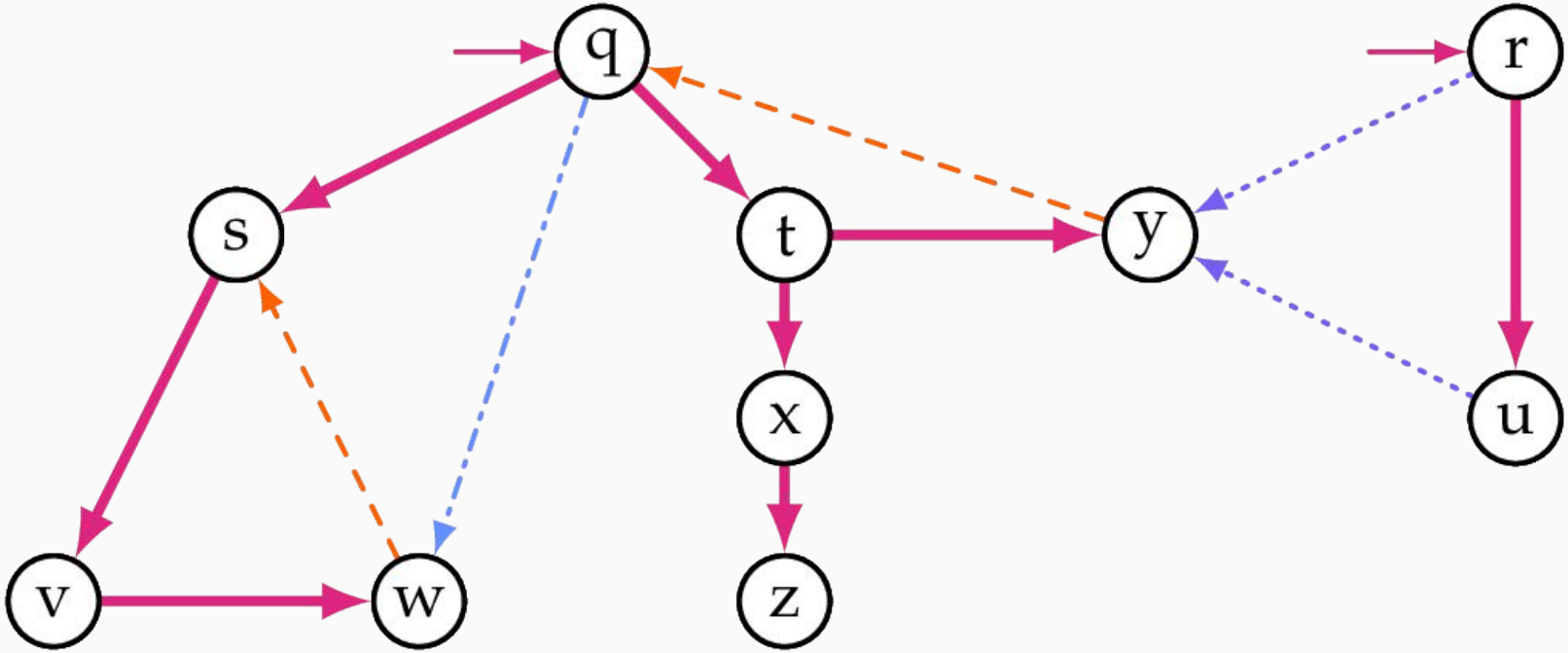




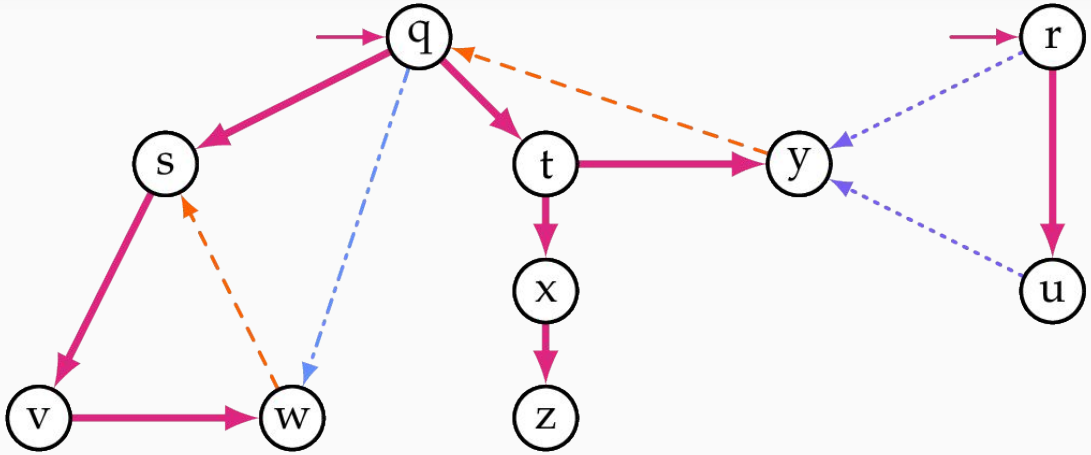
Some more examples

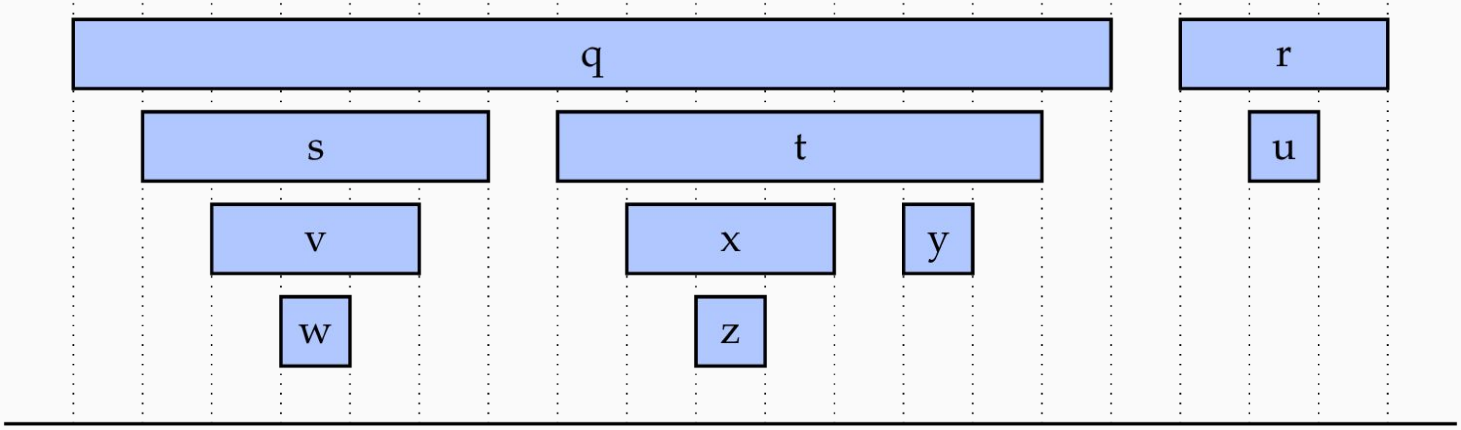


Some more examples



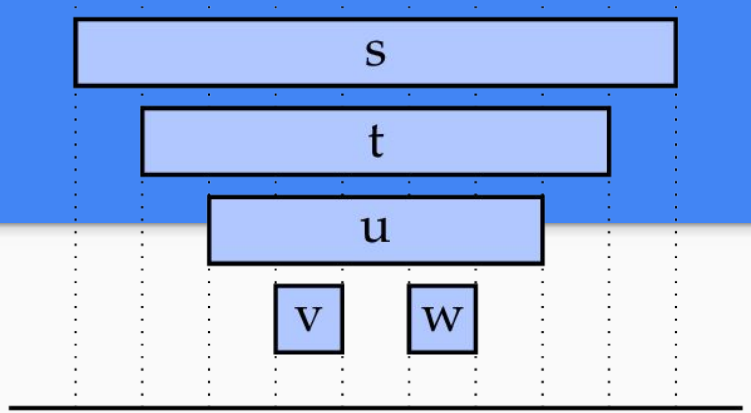
Some more examples

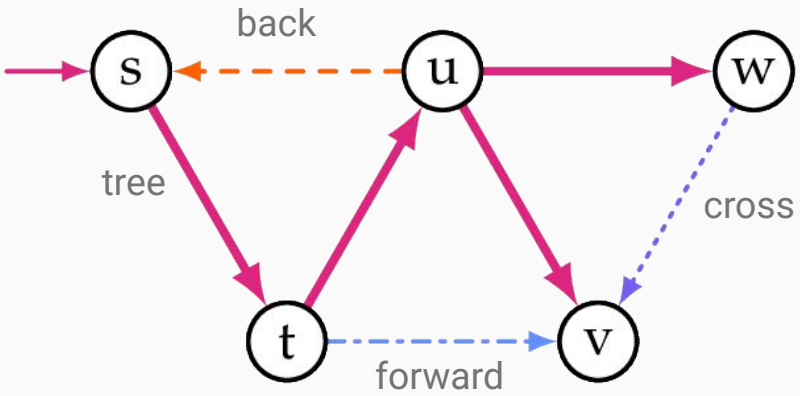




Finding a directed cycle

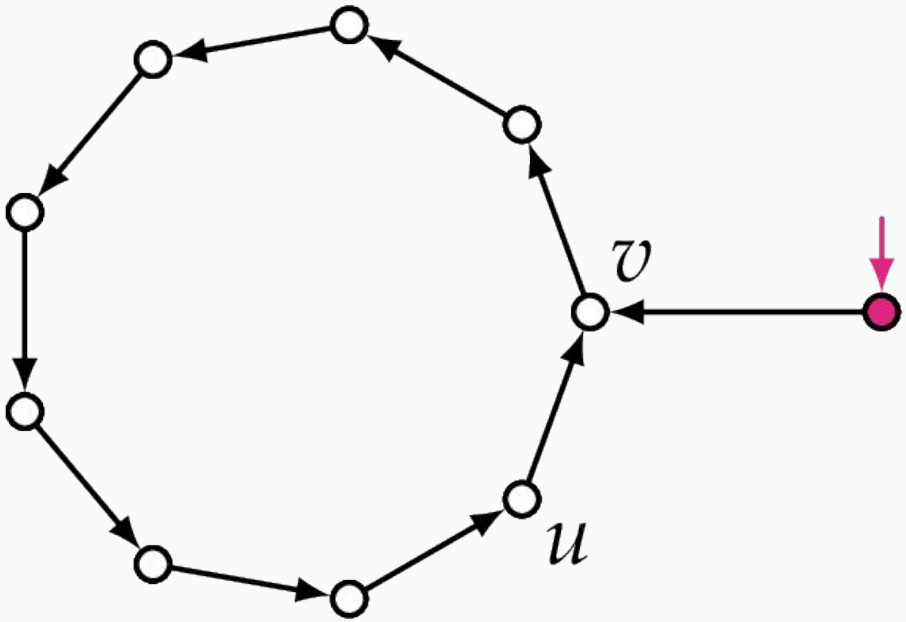
Claim: a directed cycle exists iff there is a back edge





Finding a directed cycle

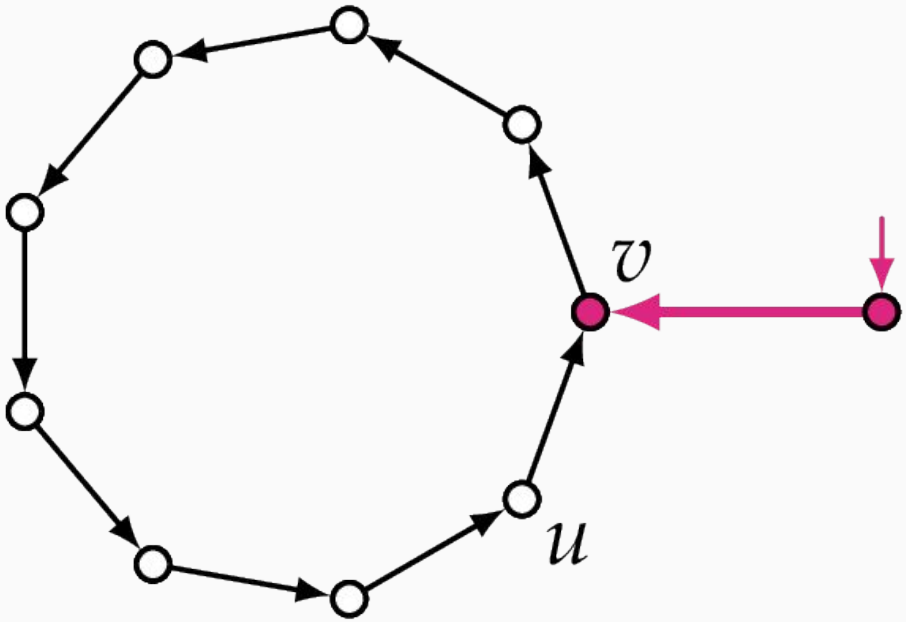
How does a DFS interact with a cycle?



Finding a directed cycle

How does a DFS interact with a cycle?

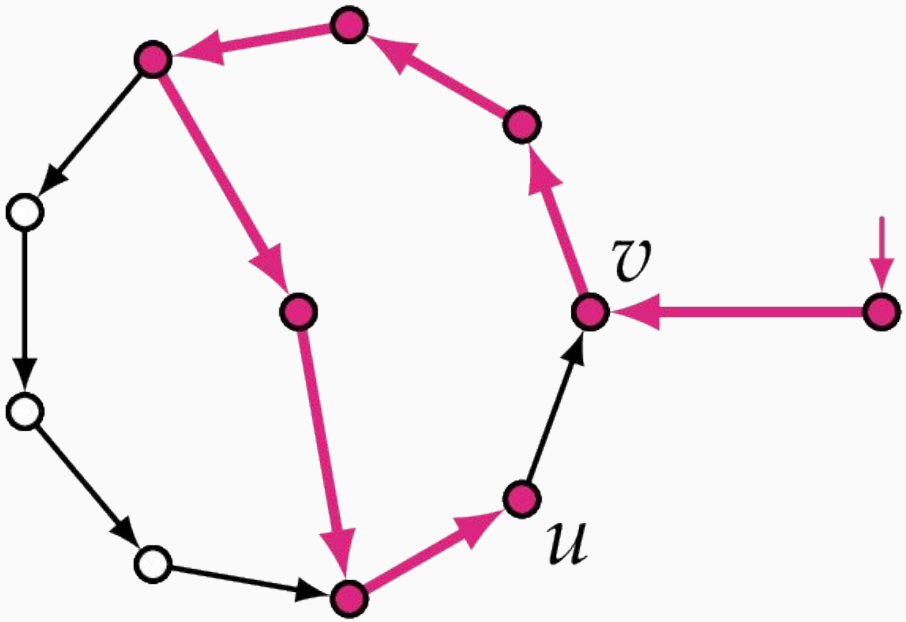
1. Visits some vertex ***v*** first



Finding a directed cycle

How does a DFS interact with a cycle?

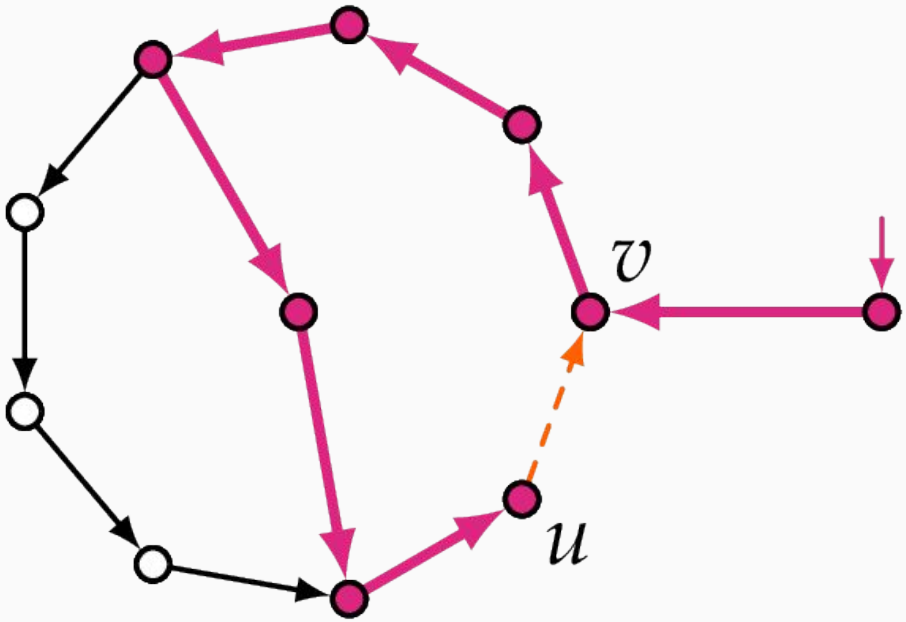
1. Visits some vertex ***v*** first
2. Eventually reaches predecessor ***u***



Finding a directed cycle

How does a DFS interact with a cycle?

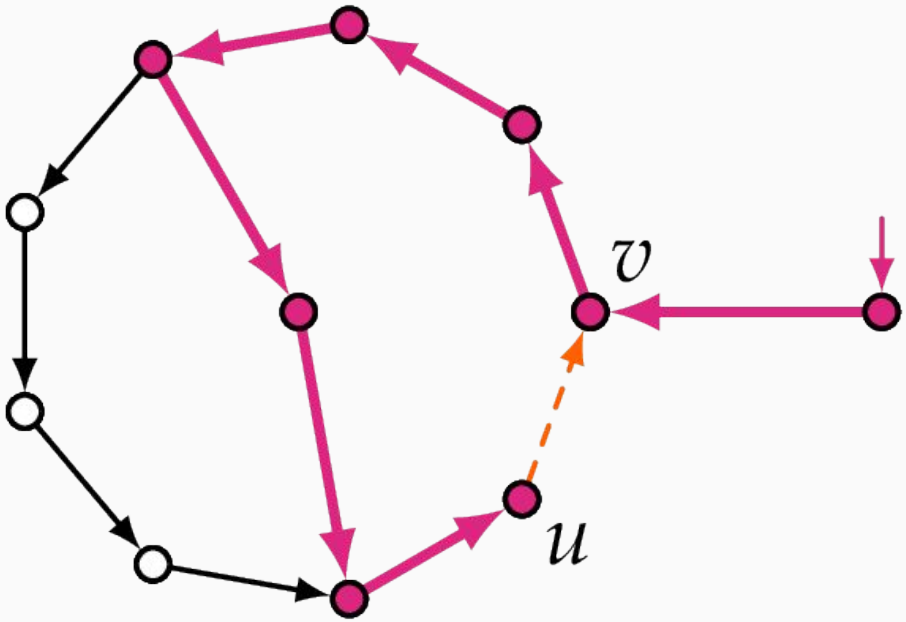
1. Visits some vertex ***v*** first
2. Eventually reaches predecessor ***u***
3. Then ***u***→***v*** is a backedge.



Finding a directed cycle

Cycle-finding algorithm:

- Do DFS.
- Check for back edges.

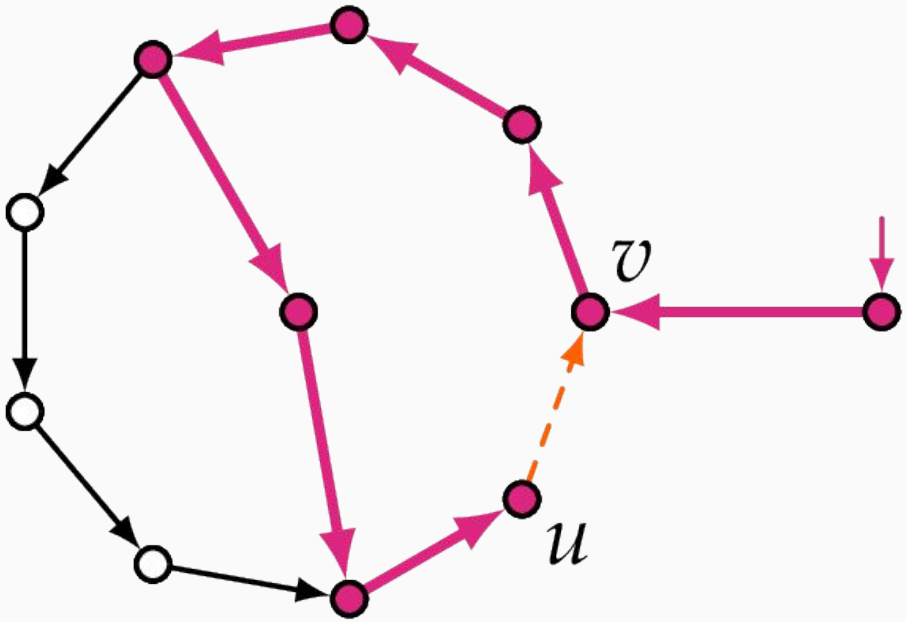


Finding a directed cycle

Cycle-finding algorithm:

- Do DFS. O(|***V***|+|***E***|)
- Check for back edges. O(|***E***|)

In total: O(|***V***|+|***E***|)



Finding a directed cycle

Cycle-finding algorithm:

- Do DFS. O(|***V***|+|***E***|)
- Check for back edges. O(|***E***|)

In total: O(|***V***|+|***E***|)

$\textsc{HasCycle}(G)$:

$\quad \textsc{DFSAll}(G)$

$\quad$ **for each** $(v \to w) \in E(G)$

$\quad\quad$ **if** $post(v) < post(w)$

$\quad\quad\quad$ **return** $\textsc{True}$

$\quad$ **return** $\textsc{False}$

Finding a directed cycle

Cycle-finding algorithm:

- Do DFS. O(|***V***|+|***E***|)
- Check for back edges. O(|***E***|)

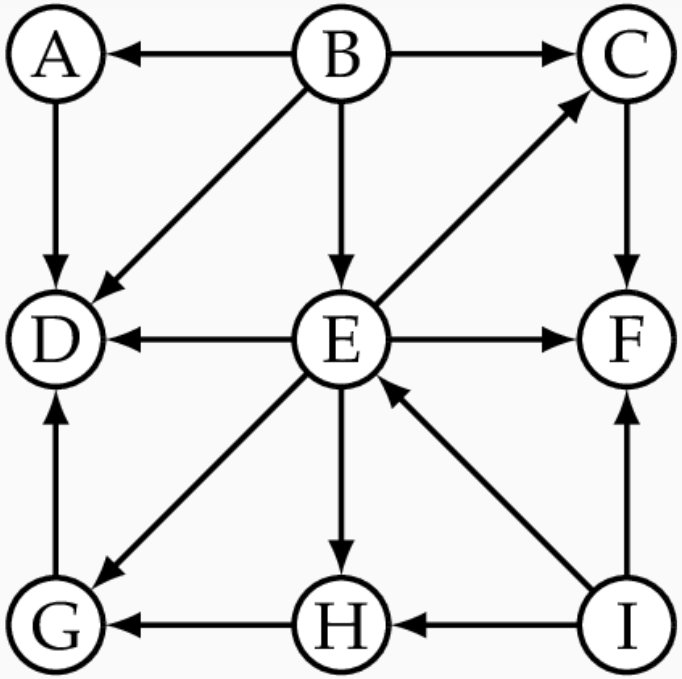
In total: O(|***V***|+|***E***|)

$$
\begin{array}{l}
\textsc{HasCycle}(G): \\
\quad \textsc{DFSAll}(G) \\
\quad \textbf{for each } (v \rightarrow w) \in E(G) \\
\quad\quad \textbf{if } post(v) < post(w) \\
\quad\quad\quad \textbf{return } \textsc{True} \\
\quad \textbf{return } \textsc{False}
\end{array}
$$

Can also modify DFSAll to find a cycle “along the way.”

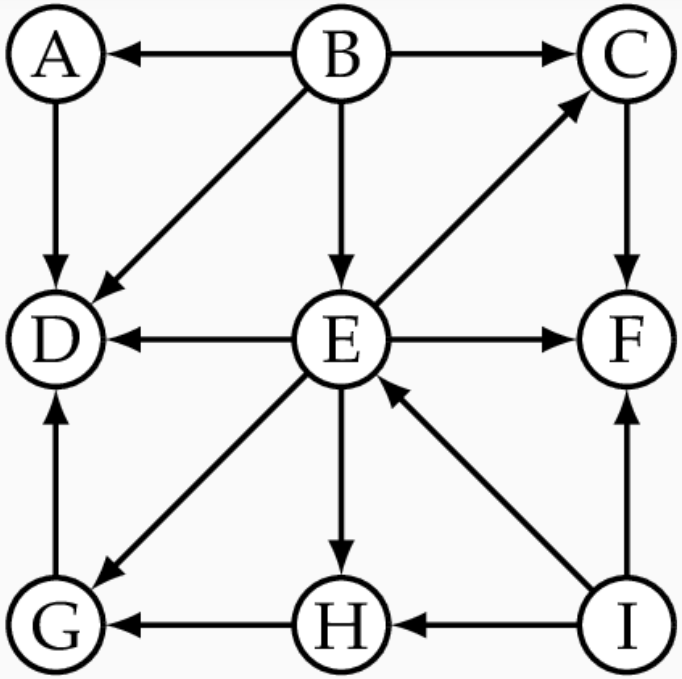
DAGs and topological sorts

A ***directed acyclic graph*** (DAG) has no directed cycles



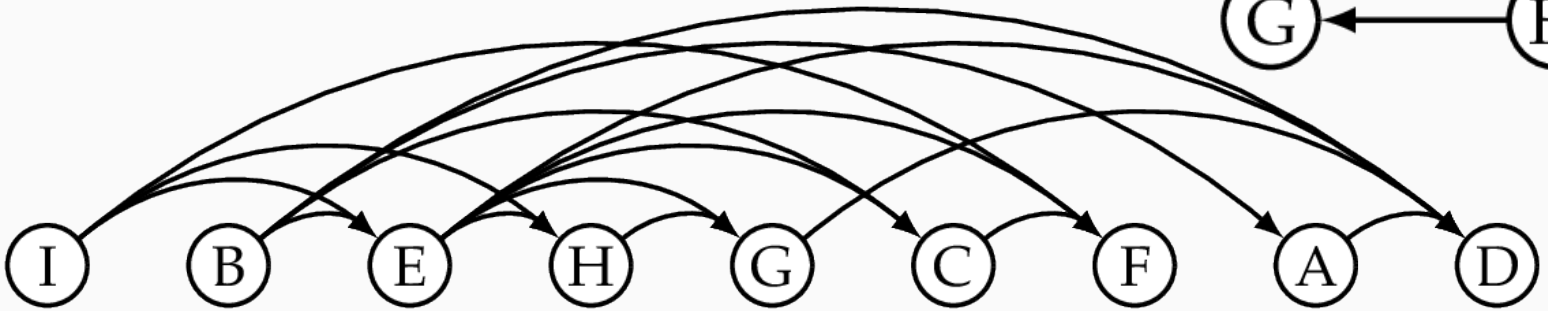
DAGs and topological sorts

A ***directed acyclic graph*** (DAG) has no directed cycles



A ***topological sort*** is an ordering of the vertices where all edges point “to the right”:

I, B, E, H, G, C, F, A, D



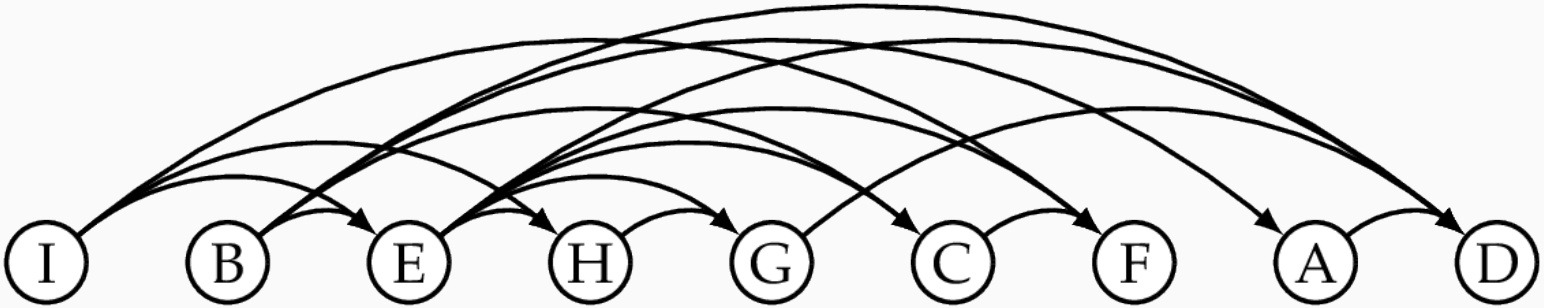
DAGs and topological sorts

Claim: A graph is a DAG iff it has a topological sort

DAGs and topological sorts

Claim: A graph is a DAG iff it has a topological sort

Easier direction: topological sort → DAG

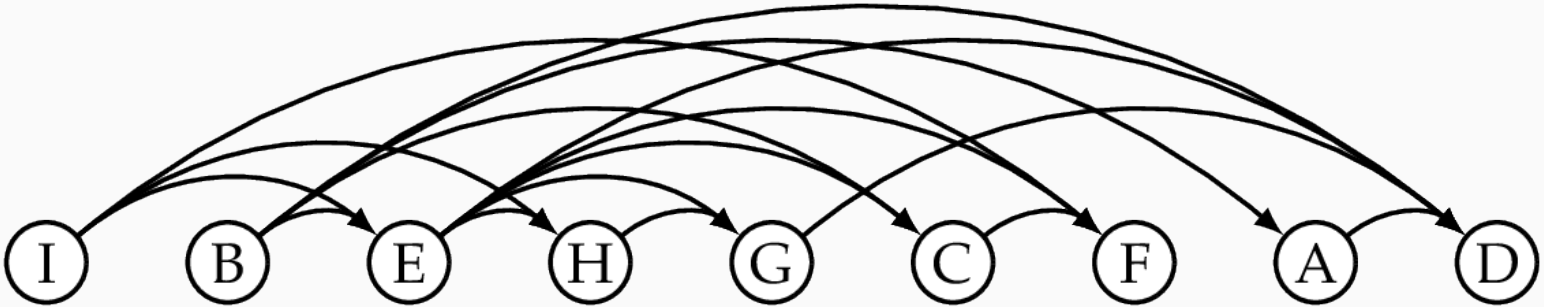


DAGs and topological sorts

Claim: A graph is a DAG iff it has a topological sort

Easier direction: topological sort → DAG

- A cycle would have a left-pointing edge



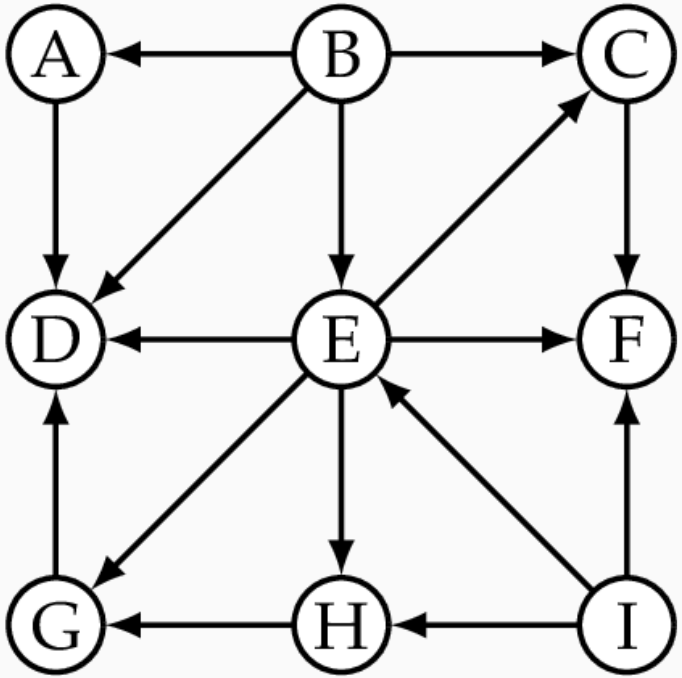
DAGs and topological sorts

Claim: A graph is a DAG iff it has a topological sort

Easier direction: topological sort → DAG

- A cycle would have a left-pointing edge

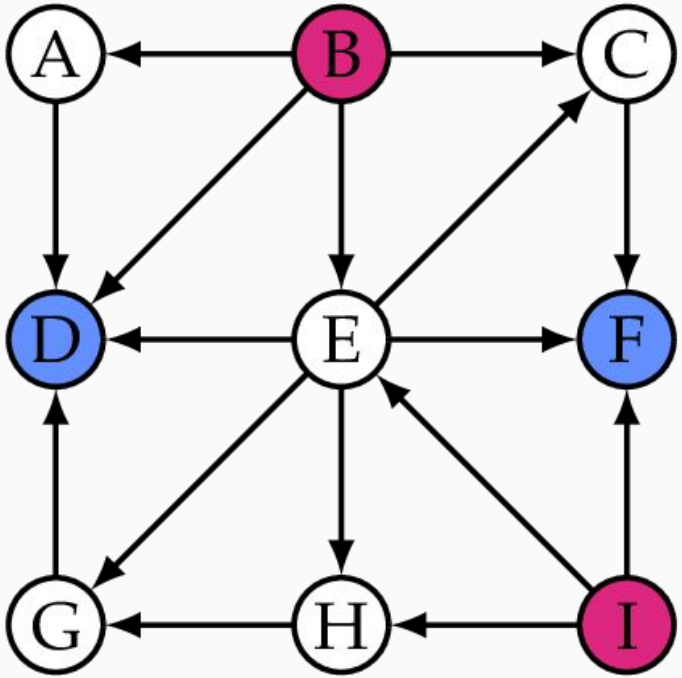
Harder direction: DAG → topological sort



Sources and sinks

Every DAG has a **source** and a **sink**.

- Source: indegree 0 (**B**, **I**)
- Sink: outdegree 0 (**D**, **F**)

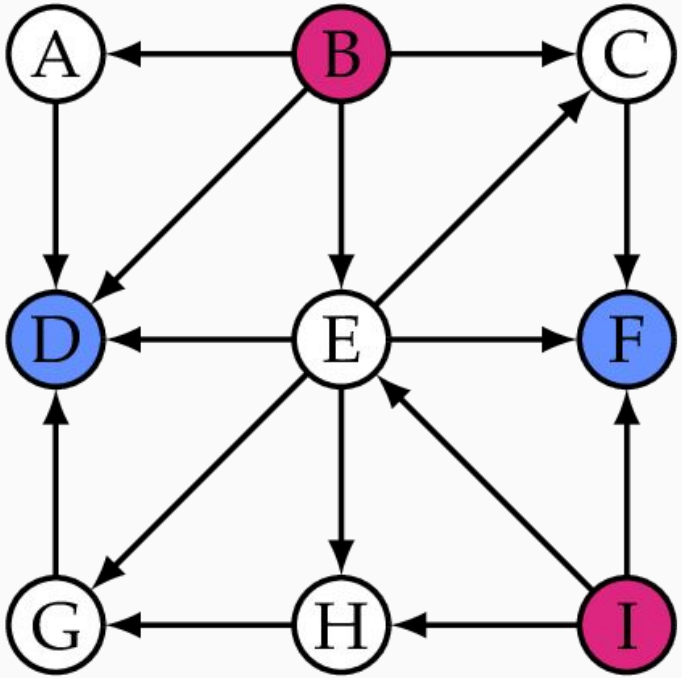


Sources and sinks

Every DAG has a **source** and a **sink**.

- Source: indegree 0 (**B**, **I**)
- Sink: outdegree 0 (**D, F**)

Proof sketch: keep walking…



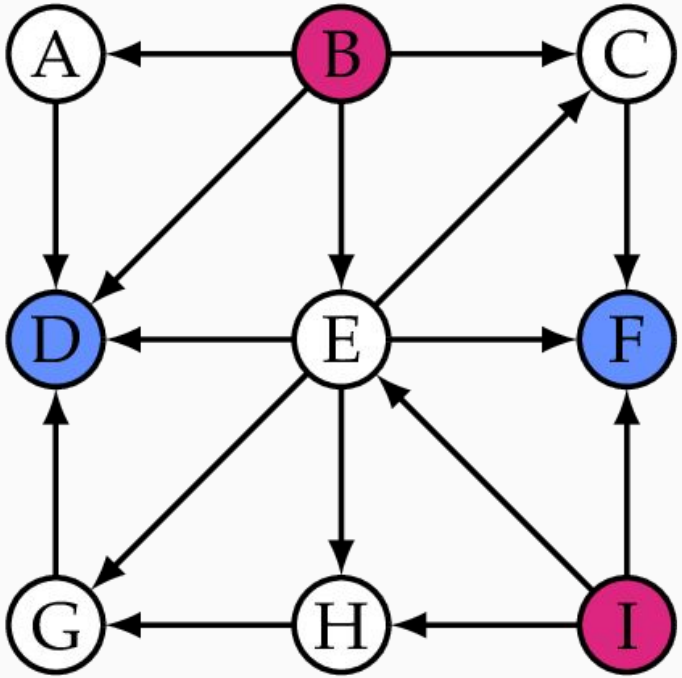
Sources and sinks

Every DAG has a **source** and a **sink**.

- Source: indegree 0 (**B**, **I**)
- Sink: outdegree 0 (**D, F**)

Proof sketch: keep walking…

Topological sort: remove a source and recurse



Information from a DFS

Recall:

- ($\boldsymbol{u} \to \boldsymbol{v}$) back edge $\to$ post($\boldsymbol{u}$) < post($\boldsymbol{v}$)

```
HasCycle(G):
    DFSAll(G)
    for each (v→w) ∈ E(G)
        if post(v) < post(w)
            return True
    return False
```

Information from a DFS

Recall:

- ($\boldsymbol{u}\rightarrow\boldsymbol{v}$) back edge $\rightarrow$ post($\boldsymbol{u}$) < post($\boldsymbol{v}$)

In a DAG, post($\boldsymbol{u}$) > post($\boldsymbol{v}$) for **every** edge

```
HasCycle(G):
    DFSAll(G)
    for each (v→w) ∈ E(G)
        if post(v) < post(w)
            return True
    return False
```

Topological sorting

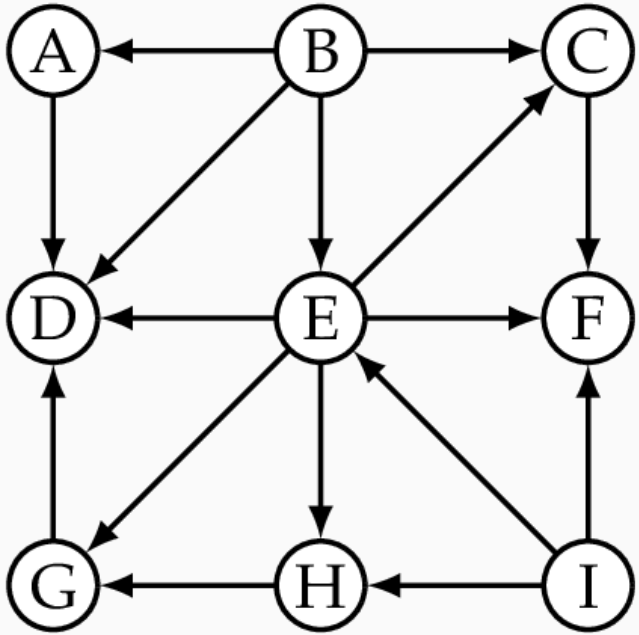
Postorder traversal is a topological sort, but backwards

Topological sort algorithm:

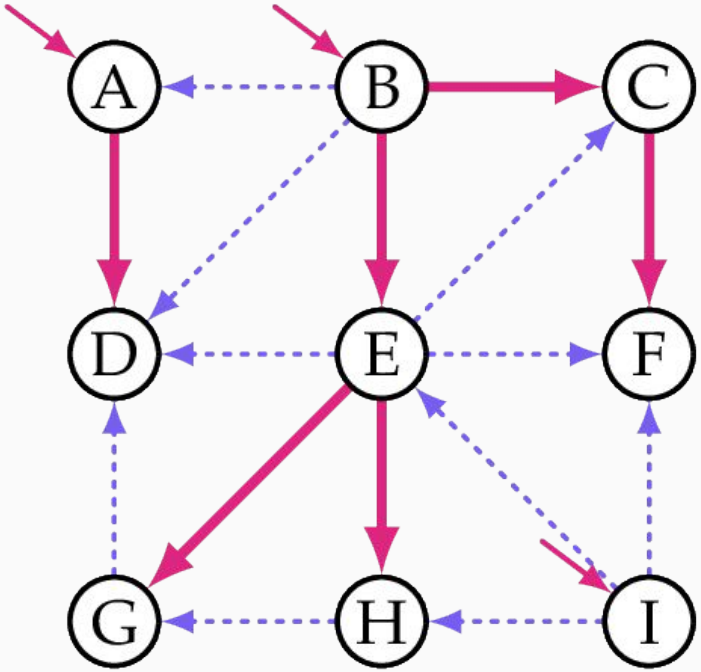
1. Do DFS.
2. Reverse postorder traversal.

```
S := STACK()
DFSALL(G):
    for each v ∈ V(G)
        if v is unmarked
            DFS(G, v)
    while |S| > 0
        print S.POP()
DFS(G, v):
    mark v
    for each (v→w) ∈ E(G)
        if w is unmarked
            DFS(G, w)
    S.PUSH(v)
```

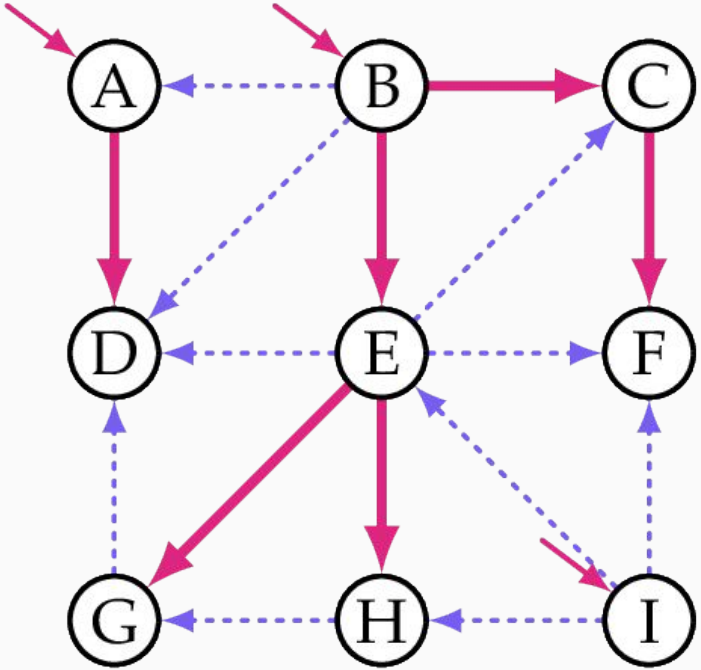
Topological sort from DFS

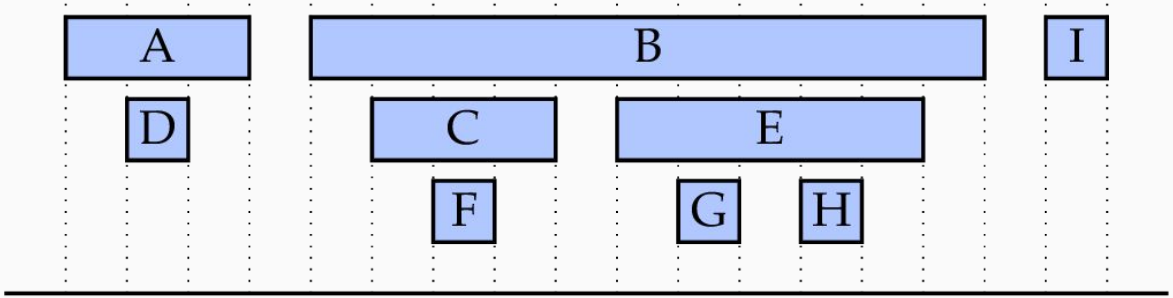


Topological sort from DFS

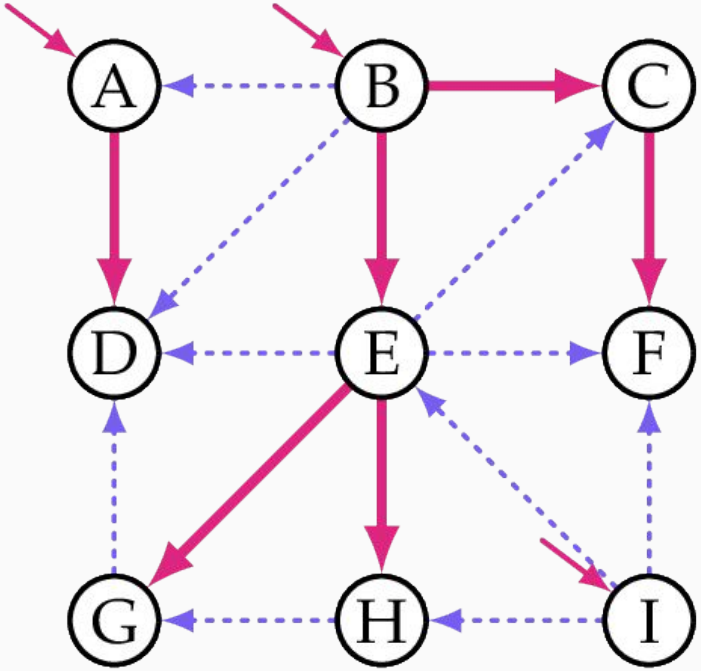


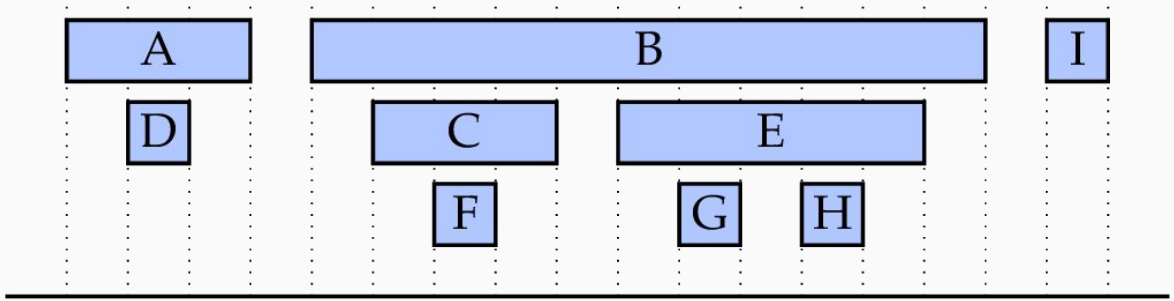
Topological sort from DFS

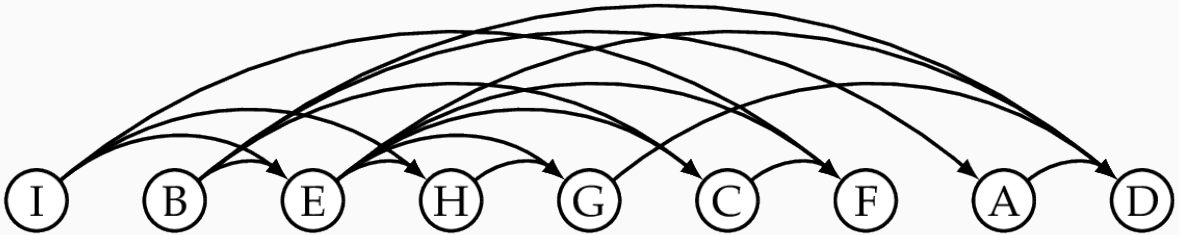




Topological sort from DFS







Topological sorting

Postorder traversal is a topological sort, but backwards

Topological sort algorithm:

1. Do DFS. O(|***V***| + |***E***|)
2. Reverse postorder traversal. O(|***V***|)

Total runtime: O(|***V***| + |***E***|)

```
S := STACK()
DFSALL(G):
    for each v ∈ V(G)
        if v is unmarked
            DFS(G, v)
    while |S| > 0
        print S.POP()
DFS(G, v):
    mark v
    for each (v→w) ∈ E(G)
        if w is unmarked
            DFS(G, w)
    S.PUSH(v)
```